# Reviewer Pack — Tropicalized Homology Groups of Knots vs BP_ReadTwice Circui…

Entry 4257ffc288bb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 06:19:40 UTC

## Tropicalized Homology Groups of Knots vs BP_ReadTwice Circuit Size

**Entry ID**: `4257ffc288bb` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 06:19:40 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebraic Topology (Homology Theory) **Field B** (complexity object): Complexity Theory: BP_ReadTwice Circuit Complexity

**Statement**:

> ['For a given knot K, the tropicalized homology groups HK(tropical) have a minimal rank that is polynomially related to the size of its BP_readtwice complexity, i.e., E[|HK(tropical)|] = Θ(log n)', 'Furthermore, for any two knots K1 and K2 with BP_readtwice complexities C(K1) and C(K2), respectively, there exists a constant C such that |HK(tropical)| ≤ C * log(C(K1)) + C * log(C(K2))']

**Rationale (proposer's reasoning)**:

> ['Homology groups capture the topological structure of knots, which might reveal nontrivial information about computational complexity due to the intricate relationships between topology and computation.', 'The tropicalization process can simplify complex algebraic structures into more tractable ones, potentially allowing for a deeper understanding of the complexity of BP_readtwice circuits.']

**Taxonomy category**: `BP_READTWICE` (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `8c70fd26bc1b8fe1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> The tropicalized homology group |HK(tropical)| for knot K is polynomially related to the BP_readtwice complexity C(K).

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | LLM₁ | LLM₂ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 1.00 | SAFE | UNCERTAIN |
| NATURAL_PROOFS | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| KARP_LIPTON | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |

## 4. Novelty audit

**Verdict**: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - `tropicalized homology groups AND knot AND BP_readtwice complexity` - `polynomial relationship AND tropicalized homology AND BP_readtwice circuit size` - `knots AND minimal rank AND θ(log n) AND BP_readtwice complexity`

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random knot of size n
    def generate_knot(n):
        return [random.randint(0, 1) for _ in range(n)]

    # Compute the tropicalized homology group |HK(tropical)|
    def tropicalized_homology(knot):
        # Simplified model: sum of elements in the knot
        return sum(knot)

    # Compute the BP_readtwice circuit size C(K)
    def bp_readtwice_complexity(knot):
        # Simplified model: length of the knot
        return len(knot)

    n_values = [5, 10, 15, 20, 30, 40]
    HK_tropical_values = []
    C_K_values = []

    for n in n_values:
        knot = generate_knot(n)
        HK_tropical = tropicalized_homology(knot)
        C_K = bp_readtwice_complexity(knot)

        if C_K == 0:
            continue

        HK_tropical_values.append(HK_tropical)
```

```
        C_K_values.append(C_K)

    if not HK_tropical_values or not C_K_values:
        return {
            "metric_name": "E[|HK(tropical)|]",
            "metric_value": None,
            "instances_tested": len(n_values),
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    # Polynomial fit to estimate E[|HK(tropical)|]
    def polynomial_fit(x, y):
        n = len(x)
        A = [[0] * (n + 1) for _ in range(n + 1)]
        B = [0] * (n + 1)

        for i in range(n):
            for j in range(n + 1):
                A[j][i] = x[i] ** j
            B[i] = y[i]

        def gaussian_elimination(A, B):
            n = len(B)
            for i in range(n):
                max_row = i
                for k in range(i+1, n):
                    if abs(A[k][i]) > abs(A[max_row][i]):
                        max_row = k
                A[i], A[max_row] = A[max_row], A[i]
                B[i], B[max_row] = B[max_row], B[i]

                factor = Fraction(A[i][i], 1)
                for j in range(i, n + 1):
                    A[i][j] /= factor
                B[i] /= factor

                for k in range(n):
                    if k != i:
                        factor = Fraction(A[k][i], 1)
                        for j in range(i, n + 1):
                            A[k][j] -= factor * A[i][j]
                        B[k] -= factor * B[i]

            return [B[i] / A[i][i] for i in range(n)]

        coefficients = gaussian_elimination(A, B)
        return coefficients

    coefficients = polynomial_fit(C_K_values, HK_tropical_values)

    # Estimate E[|HK(tropical)|] using the fitted polynomial
    def estimate_expected_value(coefficients, C_K):
        n = len(coefficients) - 1
        value = sum(coefficients[i] * (C_K ** i) for i in range(n + 1))
        return value
```

```
    expected_values = [estimate_expected_value(coefficients, C_K) for C_K in C_K_values]

    # Statistical analysis
    mean = sum(expected_values) / len(expected_values)
    std_dev = math.sqrt(sum((x - mean) ** 2 for x in expected_values) / len(expected_values))

    return {
        "metric_name": "E[|HK(tropical)|]",
        "metric_value": mean,
        "instances_tested": len(n_values),
        "conjecture_holds": all(abs(mean - value) <= std_dev for value in expected_values),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_va
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fracti
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_131a622d.py", line 125, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_131a622d.py", line 97, in run_trial
    coefficients = polynomial_fit(C_K_values, HK_tropical_values)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_131a622d.py", line 94, in polynomial_fi
    coefficients = gaussian_elimination(A, B)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_131a622d.py", line 82, in gaussian_elim
```

```
    A[i][j] /= factor
    ~~~~^^^
IndexError: list index out of range
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> The test crashed before producing data, which prevents us from verifying the polynomial relationship between the tropicalized homology group |HK(tropical)| and the BP_readtwice complexity C(K). | next: Re-run the test to ensure it completes successfully without crashing. If successful, re-evaluate the results against the pre-registered support conditions.

## 11. Audit log (LLM calls)

**Total LLM calls**: 11

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | glm4:latest | 0 | 0 | 12194 |
| 2 | propose | ollama_remote | glm4:latest | 0 | 0 | 10666 |
| 3 | propose | ollama_remote | glm4:latest | 0 | 0 | 10299 |
| 4 | preregistration | ollama_remote | glm4:latest | 0 | 0 | 5906 |
| 5 | novelty | ollama_remote | glm4:latest | 0 | 0 | 4698 |
| 6 | novelty | ollama_remote | glm4:latest | 0 | 0 | 5394 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 19390 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 11211 |
| 9 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 10236 |
| 10 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 14035 |
| 11 | judge | ollama_remote | glm4:latest | 0 | 0 | 12326 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 116354 ms total latency. Provider mix: {'ollama_remote': 11}

*(full prompt+response transcripts available in `research/audit/4257ffc288bb.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4257ffc288bb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/4257ffc288bb.tar.gz` (if generated)